

AI Contract Analyzer — Complete Line-by-Line Technical Guide

Interview Preparation Document

TABLE OF CONTENTS

1. [Project Overview & Architecture](#)
 2. [How Services Connect to Each Other](#)
 3. [Docker Compose — Orchestration Layer](#)
 4. [Database Schema \(schema.sql\)](#)
 5. [Backend — pom.xml \(Dependencies\)](#)
 6. [Backend — application.yml \(Configuration\)](#)
 7. [Backend — Main Application Class](#)
 8. [Backend — CORS Configuration](#)
 9. [Backend — AI Configuration](#)
 10. [Backend — ChatRequest DTO](#)
 11. [Backend — Upload Controller](#)
 12. [Backend — Chat Controller](#)
 13. [Backend — Document Ingestion Service](#)
 14. [Backend — RAG Service](#)
 15. [Backend — Dockerfile](#)
 16. [Frontend — package.json](#)
 17. [Frontend — Vite Configuration](#)
 18. [Frontend — index.html](#)
 19. [Frontend — main.jsx \(Entry Point\)](#)
 20. [Frontend — App.jsx \(Main Component\)](#)
 21. [Frontend — Dockerfile](#)
 22. [Complete Data Flow: How the Contract Analyzer Fetches Correct Results](#)
 23. [Key Concepts for Interview](#)
-

1. Project Overview & Architecture

What This Project Does

This is an **AI-powered contract analysis tool** that:

1. Accepts PDF contract uploads
2. Breaks them into chunks and stores them as vector embeddings in a database
3. When a user asks a question, finds the most relevant chunks using **semantic similarity search**
4. Sends those chunks as context to a local AI model (Llama3) which generates an answer
5. Streams the AI answer back to the user in real-time, token by token

Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite	User interface
Backend	Java 21 + Spring Boot 3.3 (WebFlux)	API server (reactive/non-blocking)
AI Framework	Spring AI 1.0.0-M4	Orchestrates AI model interactions
LLM (Chat)	Ollama + Llama3	Generates natural language answers
Embeddings	Ollama + nomic-embed-text	Converts text to 768-dimension vectors
Database	PostgreSQL 16 + pgvector	Stores vectors and performs similarity search
Container	Docker Compose	Runs all services together

What is RAG (Retrieval-Augmented Generation)?

RAG is a pattern where:

1. **Retrieval**: Find relevant documents from a knowledge base using vector similarity

- 2. **Augmentation:** Inject those documents into the AI prompt as context
- 3. **Generation:** The AI generates an answer based ONLY on that context

This prevents the AI from “hallucinating” (making things up) because it can only use information from your actual documents.

2. How Services Connect to Each Other

Docker Compose Network

Frontend

Backend

DB

Ollama

:3000

:8080

:5432

:11434

EXTERNAL PORTS (your browser):

- Frontend: localhost:3000

- Backend: localhost:8000 (maps to internal 8080)

- DB: localhost:5432

- Ollama: localhost:11434

Connection Details:

From	To	Connection String	Purpose
Frontend	Backend	http://localhost:8000/api/*	REST API calls
Backend	Database	jdbc:postgresql://db:5432/contractdb	Store/query vectors
Backend	Ollama	http://ollama:11434	Generate embeddings & chat responses
Docker	Database	pg_isready healthcheck	Ensure DB is ready before backend starts

Key Point: Inside Docker, services reference each other by **service name** (e.g., `db`, `ollama`), not `localhost`. Docker's internal DNS resolves these names automatically.

3. Docker Compose — Orchestration Layer

File: `docker-compose.yml`

```
version: '3.9'
```

- **Purpose:** Declares the Docker Compose file format version 3.9 (latest stable).

```
services:
```

- **Purpose:** Begins the definition of all containers (services) that will run together.
-

Service 1: Database (`db`)

```
db:  
  image: pgvector/pgvector:pg16
```

- **`db`** : Service name — other containers use this name to connect (e.g., `db:5432`)
- **`image: pgvector/pgvector:pg16`** : Uses a pre-built PostgreSQL 16 image that includes the `pgvector` extension (enables storing and searching vector embeddings)

```
  container_name: contract-db
```

- **Purpose:** Gives the container a human-readable name (shows in `docker ps`)

```
  environment:  
    POSTGRES_DB: contractdb  
    POSTGRES_USER: postgres  
    POSTGRES_PASSWORD: postgres
```

- **Purpose:** Environment variables that PostgreSQL reads on first startup to:
 - Create a database named `contractdb`
 - Set the superuser username to `postgres`
 - Set the password to `postgres`

```
ports:
  - "5432:5432"
```

- **Purpose:** Maps container port 5432 to host port 5432 (format: `host:container`). This allows you to connect from your local machine using tools like pgAdmin.

```
volumes:
  - pgdata:/var/lib/postgresql/data
  - ./db/schema.sql:/docker-entrypoint-initdb.d/schema.sql
```

- **Line 1:** Creates a named Docker volume `pgdata` stores database files persistently (survives container restarts)
- **Line 2:** Mounts the local `schema.sql` file into the container's init directory. PostgreSQL **automatically executes** any `.sql` file in `/docker-entrypoint-initdb.d/` on first startup.

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 5s
  timeout: 5s
  retries: 10
```

- **Purpose:** Docker periodically runs `pg_isready` to check if PostgreSQL is accepting connections
- **interval: 5s** : Check every 5 seconds
- **timeout: 5s** : Each check must complete within 5 seconds
- **retries: 10** : After 10 failed checks, mark as unhealthy
- **Why it matters:** The backend uses `depends_on: db: condition: service_healthy` — it won't start until this healthcheck passes

Service 2: Ollama (AI Model Server)

```
ollama:
  image: ollama/ollama:latest
  container_name: contract-ollama
  ports:
    - "11434:11434"
```

- **Purpose:** Runs Ollama, a local AI model server. Port 11434 is Ollama's default API port.

```
volumes:
  - ollama_data:/root/.ollama
```

- **Purpose:** Persists downloaded AI models (4+ GB) so they don't re-download every time you restart.

```
entrypoint: ["/bin/sh", "-c"]
command:
  - |
    ollama serve &
    sleep 10
    ollama pull nomic-embed-text
    ollama pull llama3
    wait
```

- **entrypoint** : Overrides the default container startup command to run a shell script
- **ollama serve &** : Starts the Ollama server in the background (**&** = background process)
- **sleep 10** : Waits 10 seconds for the server to be ready
- **ollama pull nomic-embed-text** : Downloads the embedding model (~270MB) — converts text to 768-dimension vectors
- **ollama pull llama3** : Downloads the chat model (~4GB) — generates natural language responses
- **wait** : Keeps the container running (waits for background processes to finish)

Service 3: Backend (Spring Boot)

```
backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
```

- **Purpose:** Instead of using a pre-built image, Docker **builds** the backend from the `./backend` directory using its Dockerfile.

```
container_name: contract-backend
ports:
  - "8000:8080"
```

- **Purpose:** Maps internal port 8080 (Spring Boot default) to external port 8000. Your browser hits `localhost:8000`.

```
environment:
  SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/contractdb
  SPRING_DATASOURCE_USERNAME: postgres
  SPRING_DATASOURCE_PASSWORD: postgres
  SPRING_AI_OLLAMA_BASE_URL: http://ollama:11434
  SPRING_AI_OLLAMA_CHAT_MODEL: llama3
  SPRING_AI_OLLAMA_EMBEDDING_MODEL: nomic-embed-text
```

- **Purpose:** Passes configuration to Spring Boot via environment variables. Spring Boot automatically picks these up and overrides `application.yml` values.
- **db:5432** : Uses Docker's internal DNS — `db` resolves to the database container's IP
- **ollama:11434** : Same — `ollama` resolves to the Ollama container's IP

```
depends_on:
  db:
    condition: service_healthy
  ollama:
    condition: service_started
```

- **Purpose:** Start order control:
 - Wait until DB healthcheck passes (`service_healthy`) — guarantees DB is ready

- Wait until Ollama container starts (`service_started`) — doesn't guarantee models are downloaded yet
-

Service 4: Frontend (React)

```
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: contract-frontend
  ports:
    - "3000:3000"
  depends_on:
    - backend
```

- **Purpose:** Builds the React app, serves it on port 3000, waits for backend to start first.
-

Named Volumes

```
volumes:
  pgdata:
  ollama_data:
```

- **Purpose:** Declares persistent storage volumes. Data survives `docker-compose down` (but not `docker-compose down -v`).
-

4. Database Schema (schema.sql)

File: `db/schema.sql`

This file runs automatically when PostgreSQL starts for the first time.

```
CREATE EXTENSION IF NOT EXISTS vector;
```

- **Purpose:** Enables the `pgvector` extension — adds the `vector` data type and similarity search operators to PostgreSQL.
- **IF NOT EXISTS** : Safe to run multiple times without errors.

```
CREATE TABLE IF NOT EXISTS contracts (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL,
  filename VARCHAR(512),
  uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- **Purpose:** Metadata table tracking uploaded contracts
- **BIGSERIAL** : Auto-incrementing 64-bit integer (1, 2, 3...)
- **PRIMARY KEY** : Unique identifier for each row
- **BIGINT NOT NULL** : 64-bit integer, cannot be null — tracks which user owns the contract
- **VARCHAR(512)** : String up to 512 characters for the filename
- **DEFAULT CURRENT_TIMESTAMP** : Auto-fills with the current date/time on insert

```
CREATE TABLE IF NOT EXISTS vector_store (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  content TEXT NOT NULL,
  metadata JSONB DEFAULT '{}',
  embedding vector(768)
);
```

- **Purpose:** This is the **core table** — stores document chunks with their vector embeddings
- **UUID** : Universally unique identifier (e.g., `a1b2c3d4-e5f6-...`) — generated randomly
- **content TEXT** : The actual text content of the document chunk
- **metadata JSONB** : JSON data storing `contract_id`, `user_id`, and other attributes. JSONB is binary JSON — fast for queries.
- **embedding vector(768)** : A 768-dimensional floating-point vector — the mathematical representation of the text content. `nomic-embed-text` produces exactly 768 dimensions.

Why 768? The `nomic-embed-text` model outputs vectors with 768 numbers. Each number represents a “meaning dimension” — semantically similar texts have similar vectors.

```
CREATE INDEX IF NOT EXISTS vector_store_embedding_idx
ON vector_store USING hnsw (embedding vector_cosine_ops);
```

- **Purpose:** Creates an **HNSW index** for fast approximate nearest neighbor search
- **HNSW** : Hierarchical Navigable Small World — a graph-based algorithm for fast vector similarity search (much faster than scanning every row)
- **vector_cosine_ops** : Uses cosine similarity as the distance metric (measures angle between vectors — 1.0 = identical, 0.0 = unrelated)

```
CREATE INDEX IF NOT EXISTS vector_store_metadata_idx
ON vector_store USING gin (metadata jsonb_path_ops);
```

- **Purpose:** Creates a GIN (Generalized Inverted Index) on the JSONB metadata column
- **Why:** Enables fast filtering by `contract_id` and `user_id` without scanning every row

```
INSERT INTO contracts (id, user_id, filename, uploaded_at)
VALUES (1, 101, 'sample-contract.pdf', CURRENT_TIMESTAMP)
ON CONFLICT (id) DO NOTHING;
```

- **Purpose:** Seeds a default contract entry so you can immediately test without uploading first
- **ON CONFLICT (id) DO NOTHING** : If ID 1 already exists, skip the insert (prevents errors on restart)

5. Backend — pom.xml (Dependencies)

File: `backend/pom.xml`

Maven POM (Project Object Model) defines the project's dependencies, build configuration, and metadata.

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.3.5</version>
```

```
<relativePath/>
</parent>
```

- **Purpose:** Inherits Spring Boot 3.3.5's default configurations, dependency versions, and plugin settings. This means you don't have to specify versions for most Spring dependencies.

```
<properties>
  <java.version>21</java.version>
  <spring-ai.version>1.0.0-M4</spring-ai.version>
</properties>
```

- **Purpose:** Declares variables reused throughout the file. Java 21 enables modern features (records, text blocks, pattern matching).

Key Dependencies Explained:

Dependency	Purpose
spring-boot-starter-webflux	Reactive web framework (non-blocking I/O, supports streaming)
spring-ai-ollama-spring-boot-starter	Auto-configures connection to Ollama for chat and embeddings
spring-ai-pgvector-store-spring-boot-starter	Auto-configures PgVectorStore for storing/searching embeddings
spring-ai-tika-document-reader	Apache Tika PDF reader — extracts text from PDFs
postgresql	JDBC driver for PostgreSQL connectivity
spring-boot-starter-jdbc	Provides JdbcTemplate for raw SQL queries
jackson-databind	JSON serialization/deserialization

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>${spring-ai.version}</version>
```

```
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>
```

- **Purpose:** A BOM (Bill of Materials) manages all Spring AI dependency versions centrally. You don't need to specify versions for individual Spring AI modules.

```
<repositories>
  <repository>
    <id>spring-milestones</id>
    <url>https://repo.spring.io/milestone</url>
  </repository>
</repositories>
```

- **Purpose:** Spring AI 1.0.0-M4 is a milestone release (not yet in Maven Central), so we add the Spring milestone repository.
-

6. Backend — application.yml (Configuration)

File: `backend/src/main/resources/application.yml`

```
server:
  port: 8080
```

- **Purpose:** Spring Boot listens on port 8080 (inside the container).

```
spring:
  datasource:
    url: ${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/contractdb}
    username: ${SPRING_DATASOURCE_USERNAME:postgres}
    password: ${SPRING_DATASOURCE_PASSWORD:postgres}
    driver-class-name: org.postgresql.Driver
```

- **Purpose:** Database connection configuration
- **\${VARIABLE:default}** : If the environment variable exists, use it; otherwise use the default value after the colon
- **jdbc:postgresql://localhost:5432/contractdb** : JDBC URL format — protocol://host:port/database
- **driver-class-name** : Tells Spring which JDBC driver class to use for PostgreSQL

How DB connection works: Spring Boot auto-creates a `DataSource` bean (connection pool) using these properties. Spring AI's `PgVectorStore` and `JdbcTemplate` both use this `DataSource`.

```
ai:
  ollama:
    base-url: ${SPRING_AI_OLLAMA_BASE_URL:http://localhost:11434}
    chat:
      options:
        model: ${SPRING_AI_OLLAMA_CHAT_MODEL:llama3}
        temperature: 0.2
    embedding:
      options:
        model: ${SPRING_AI_OLLAMA_EMBEDDING_MODEL:nomic-embed-text}
```

- **base-url** : Where Ollama API is running
- **chat.options.model: llama3** : The LLM used for generating answers
- **temperature: 0.2** : Controls randomness (0.0 = deterministic, 1.0 = creative). Low temperature = more factual answers.
- **embedding.options.model: nomic-embed-text** : Model used to convert text 768-dim vectors

```
vectorstore:
  pgvector:
    index-type: hnsw
    distance-type: cosine_distance
    dimensions: 768
    schema-validation: false
    initialize-schema: false
```

- **index-type: hnsw** : Use HNSW algorithm for vector indexing
- **distance-type: cosine_distance** : Measure similarity using cosine distance
- **dimensions: 768** : Vector size must match the embedding model output

- **schema-validation: false** : Don't validate the existing table schema
 - **initialize-schema: false** : Don't auto-create tables (we handle this in schema.sql)
-

7. Backend — Main Application Class

File: **ContractAnalyserApplication.java**

```
package com.contract.analyser;
```

- **Purpose:** Java package declaration — organizes classes into namespaces. Must match the directory structure.

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

- **Purpose:** Import statements bring external classes into scope.

```
@SpringBootApplication
```

- **Purpose:** A meta-annotation combining three things:
 1. **@Configuration** — This class can define beans
 2. **@EnableAutoConfiguration** — Spring Boot auto-configures beans based on classpath dependencies
 3. **@ComponentScan** — Scans this package and sub-packages for **@Component** , **@Service** , **@Controller** , etc.

```
public class ContractAnalyserApplication {
    public static void main(String[] args) {
        SpringApplication.run(ContractAnalyserApplication.class, args);
    }
}
```

- **Purpose:** Standard Java entry point. **SpringApplication.run()** bootstraps the entire Spring context — creates beans, starts the web server, wires dependencies.
-

8. Backend — CORS Configuration

File: `config/CorsConfig.java`

```
@Configuration
```

- **Purpose:** Tells Spring this class defines beans (objects managed by Spring's IoC container).

```
@Bean
```

```
public CorsWebFilter corsWebFilter() {
```

- **@Bean** : The method's return value becomes a Spring-managed singleton bean
- **CorsWebFilter** : A WebFlux-specific filter (not MVC's `CorsFilter`) — intercepts every HTTP request to add CORS headers

```
CorsConfiguration config = new CorsConfiguration();  
config.setAllowedOrigins(List.of("http://localhost:3000", "http://frontend:3000"));
```

- **Purpose:** Defines which origins (frontend URLs) can make cross-origin requests. Browsers block requests from different origins by default (CORS policy).
- **Why two origins:** `localhost:3000` for local development, `frontend:3000` for Docker-internal requests.

```
config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
```

- **Purpose:** Which HTTP methods are allowed cross-origin. `OPTIONS` is needed for preflight requests.

```
config.setAllowedHeaders(List.of("*"));
```

- **Purpose:** Allow any request header (Content-Type, Authorization, etc.)

```
config.setAllowCredentials(true);  
config.setMaxAge(3600L);
```

- **setAllowCredentials(true)** : Allow cookies/auth tokens in cross-origin requests

- **setMaxAge(3600L)** : Browser caches the CORS preflight response for 1 hour (3600 seconds)

```
UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
source.registerCorsConfiguration("/**", config);
```

- **/**** : Apply this CORS config to ALL URL paths
-

9. Backend — AI Configuration

File: **config/AiConfig.java**

```
@Configuration
public class AiConfig {

    @Bean
    public ChatClient chatClient(OllamaChatModel chatModel) {
        return ChatClient.builder(chatModel).build();
    }
}
```

- **OllamaChatModel chatModel** : Spring AI auto-creates this bean based on `application.yml` settings. It knows how to communicate with Ollama's API.
 - **ChatClient.builder(chatModel).build()** : Creates a fluent ChatClient — a high-level API for building prompts and streaming responses.
 - **Why a @Bean?**: So other classes (like RagService) can inject `ChatClient` via constructor injection.
-

10. Backend — ChatRequest DTO

File: **dto/ChatRequest.java**

```
public record ChatRequest(
    @JsonProperty("contract_id") Long contractId,
    @JsonProperty("user_id") Long userId,
```

```
String question
) {}
```

- **record** : Java 16+ feature — immutable data class. Automatically generates constructor, getters, equals(), hashCode(), toString().
- **@JsonProperty("contract_id")** : Maps JSON field `contract_id` (snake_case) to Java field `contractId` (camelCase) during deserialization.
- **Purpose**: Represents the incoming JSON request body for the chat endpoint:

```
{"contract_id": 1, "user_id": 101, "question": "What are the terms?"}
```

11. Backend — Upload Controller

File: `controller/UploadController.java`

```
@RestController
@RequestMapping("/api")
```

- **@RestController** : Combines `@Controller` + `@ResponseBody` — every method returns data (JSON), not a view.
- **@RequestMapping("/api")** : All endpoints in this class start with `/api`.

```
private final DocumentIngestionService ingestionService;

public UploadController(DocumentIngestionService ingestionService) {
    this.ingestionService = ingestionService;
}
```

- **Purpose**: Constructor injection — Spring auto-wires the `DocumentIngestionService` bean. This is the recommended DI pattern (no `@Autowired` needed when there's only one constructor).

```
@PostMapping(value = "/upload", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
public Mono<Map<String, Object>> uploadDocument(@RequestPart("file") FilePart fileP
```

- **@PostMapping("/upload")** : Handles POST requests to `/api/upload`

- **consumes = MULTIPART_FORM_DATA_VALUE** : Expects `multipart/form-data` content type (used for file uploads)
- **Mono<Map<String, Object>>** : Reactive return type — a single async result containing a JSON map
- **@RequestPart("file") FilePart filePart** : Extracts the file from the multipart form field named "file". `FilePart` is WebFlux's reactive file upload type.

```
return Mono.fromCallable(() -> ingestionService.ingestDocument(filePart))
    .subscribeOn(Schedulers.boundedElastic());
```

- **Mono.fromCallable(...)** : Wraps a blocking operation in a reactive Mono
- **subscribeOn(Schedulers.boundedElastic())** : Executes the blocking work on an elastic thread pool (not the event loop). This is critical in WebFlux — blocking the event loop would freeze ALL requests.

12. Backend — Chat Controller

File: `controller/ChatController.java`

```
@PostMapping(value = "/chat/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<String> streamChat(@RequestBody ChatRequest request) {
    return ragService.streamResponse(request);
}
```

- **produces = TEXT_EVENT_STREAM_VALUE** : Sets response header `Content-Type: text/event-stream` — tells the browser this is a Server-Sent Events (SSE) stream.
- **Flux<String>** : Reactive type representing 0-to-N values emitted over time. Each emitted string is one token from the AI model.
- **@RequestBody ChatRequest request** : Deserializes the JSON request body into a `ChatRequest` record.

How SSE streaming works:

1. Client sends POST request
2. Server keeps the connection open
3. Server sends data as `data: token1\n\ndata: token2\n\n` format
4. Client reads tokens as they arrive (real-time streaming)

13. Backend — Document Ingestion Service

File: `service/DocumentIngestionService.java`

```
@Service
public class DocumentIngestionService {
```

- **@Service** : Stereotype annotation — marks this as a business logic bean. Spring scans and creates a singleton instance.

```
private static final Long DEFAULT_CONTRACT_ID = 1L;
private static final Long DEFAULT_USER_ID = 101L;
```

- **Purpose**: Hard-coded values for the default contract (matches the seed data in `schema.sql`).

```
private final VectorStore vectorStore;
private final JdbcTemplate jdbcTemplate;

public DocumentIngestionService(VectorStore vectorStore, JdbcTemplate jdbcTemplate) {
    this.vectorStore = vectorStore;
    this.jdbcTemplate = jdbcTemplate;
}
```

- **VectorStore** : Spring AI abstraction — auto-configured as `PgVectorStore` by the `pgvector` starter. Handles embedding generation + storage.
- **JdbcTemplate** : Spring's utility for executing raw SQL queries.

The `ingestDocument` Method — Step by Step:

```
tempFile = Files.createTempFile("upload-", "-" + filePart.filename());
File file = tempFileToFile();
filePart.transferTo(file).block();
```

- **Step 1**: Create a temporary file on disk
- **Step 2**: Transfer the uploaded content from the reactive stream to the file

- **.block()** : Blocks until the transfer completes (acceptable here because we're on a bounded-elastic thread)

```
TikaDocumentReader reader = new TikaDocumentReader(new FileSystemResource(file))
List<Document> documents = reader.get();
```

- **Step 2:** Apache Tika reads the PDF and extracts all text content. Returns a list of **Document** objects (usually one per file, containing all pages).

```
TokenTextSplitter splitter = new TokenTextSplitter(1000, 200, 5, 10000, true);
List<Document> chunks = splitter.apply(documents);
```

- **Step 3:** Splits the large document into smaller chunks:
 - **1000:** Target chunk size (in tokens/characters)
 - **200:** Overlap between consecutive chunks (ensures context isn't lost at boundaries)
 - **5:** Minimum chunk size
 - **10000:** Maximum chunk size
 - **true:** Keep separator characters

Why chunk? Embedding models have input limits, and smaller chunks give more precise similarity matches.

```
for (Document chunk : chunks) {
    chunk.getMetadata().put("contract_id", DEFAULT_CONTRACT_ID);
    chunk.getMetadata().put("user_id", DEFAULT_USER_ID);
}
```

- **Step 4:** Attach metadata to each chunk. This metadata is stored in the JSONB column and used later for filtering (multi-tenant isolation).

```
jdbcTemplate.update(
    "DELETE FROM vector_store WHERE metadata->>'contract_id' = ?",
    String.valueOf(DEFAULT_CONTRACT_ID)
);
```

- **Step 5:** Delete any existing vectors for this contract (replace, not append). The `->>'contract_id'` syntax extracts a text value from JSONB.

```
vectorStore.add(chunks);
```

- **Step 6:** This single line does THREE things:
 1. Sends each chunk's text to Ollama's `omic-embed-text` model
 2. Receives back a 768-dimension vector for each chunk
 3. INSERTs the chunk (content + metadata + embedding) into the `vector_store` table

```
finally {  
    if (tempFile != null) {  
        Files.deleteIfExists(tempFile);  
    }  
}
```

- **Purpose:** Cleanup — always delete the temporary file, even if an error occurred.

14. Backend — RAG Service (The Core Intelligence)

File: `service/RagService.java`

This is the **heart of the application** — implements the RAG pattern.

```
private static final String SYSTEM_PROMPT_TEMPLATE = ""  
    You are a helpful AI contract analyst...  
    CONTEXT FROM CONTRACT:  
    {context}  
    "";
```

- **Purpose:** A text block (Java 15+) defining the system prompt template. The `{context}` placeholder gets replaced with actual document chunks.
- **Guardrail:** The prompt explicitly tells the AI to respond "I cannot find that information in the contract." if the context doesn't contain the answer.

```
public Flux<String> streamResponse(ChatRequest request) {  
    return Flux.defer(() -> {
```

- **Flux.defer(...)** : Lazily creates the Flux — the code inside only runs when someone subscribes (when the HTTP response starts streaming).

```
FilterExpressionBuilder builder = new FilterExpressionBuilder();
var filterExpression = builder.and(
    builder.eq("contract_id", request.contractId()),
    builder.eq("user_id", request.userId())
).build();
```

- **Purpose:** Builds a metadata filter expression: `contract_id == 1 AND user_id == 101`
- **Why:** Multi-tenant isolation — even if the database has vectors from multiple contracts/users, each query only sees its own data.

```
SearchRequest searchRequest = SearchRequest.query(request.question())
    .withTopK(3)
    .withFilterExpression(filterExpression);
```

- **Purpose:** Configures the similarity search:
 - **.query(question)** : The user's question gets embedded into a vector, then compared against stored vectors
 - **.withTopK(3)** : Return only the 3 most similar chunks
 - **.withFilterExpression(...)** : Apply the metadata filter BEFORE similarity comparison

```
List<Document> relevantDocs = vectorStore.similaritySearch(searchRequest);
```

- **What happens internally:**
 1. Sends the question to Ollama's `omic-embed-text` gets a 768-dim query vector
 2. PostgreSQL uses the HNSW index to find the 3 closest vectors (by cosine similarity)
 3. Filters results by metadata (`contract_id`, `user_id`)
 4. Returns the matching Document objects with their text content

```
String context = relevantDocs.stream()
    .map(Document::getContent)
    .collect(Collectors.joining("\n\n---\n\n\n"));
```

- **Purpose:** Joins the 3 relevant text chunks into a single context string, separated by `---`.

```
if (context.isBlank()) {
    return Flux.just("I cannot find that information in the contract.");
}
```

- **Purpose:** If no relevant documents were found, return the fallback message immediately.

```
String systemPrompt = SYSTEM_PROMPT_TEMPLATE.replace("{context}", context);
```

- **Purpose:** Inject the retrieved context into the system prompt template.

```
return chatClient.prompt()
    .system(systemPrompt)
    .user(request.question())
    .stream()
    .content();
```

- **Purpose:** Uses Spring AI's fluent ChatClient API:
 - **.prompt()** : Start building a prompt
 - **.system(systemPrompt)** : Set the system message (instructions + context)
 - **.user(question)** : Set the user's question
 - **.stream()** : Stream the response (not wait for the full answer)
 - **.content()** : Extract just the text content as a `Flux<String>` (each element = one token)

```
}).subscribeOn(Schedulers.boundedElastic());
```

- **Purpose:** Run the blocking vector search on an elastic thread (not the event loop).

15. Backend — Dockerfile

File: `backend/Dockerfile`

FROM eclipse-temurin:21-jdk AS build

- **Stage 1:** Uses Eclipse Temurin JDK 21 image (full JDK with compiler) for building.
- **AS build** : Names this stage “build” for reference in the next stage.

WORKDIR /app

COPY . .

RUN chmod +x mvnw && ./mvnw clean package -DskipTests

- **WORKDIR /app** : Set working directory inside container
- **COPY . .** : Copy all backend source files into the container
- **chmod +x mvnw** : Make Maven wrapper executable
- **./mvnw clean package -DskipTests** : Build the JAR file (skip tests for faster build)

FROM eclipse-temurin:21-jre

- **Stage 2:** Uses a smaller JRE-only image (no compiler needed at runtime). This reduces the final image size significantly.

WORKDIR /app

COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]

- **COPY --from=build** : Copy the built JAR from Stage 1 into Stage 2
- **EXPOSE 8080** : Documentation — declares which port the app listens on
- **ENTRYPOINT** : The command that runs when the container starts

Multi-stage build benefit: Final image is ~300MB (JRE only) instead of ~700MB (full JDK + Maven cache).

16. Frontend — package.json

File: frontend/package.json

```
{  
  "name": "contract-analyser-frontend",
```

```
"private": true,  
"version": "1.0.0",  
"type": "module",
```

- **"private": true** : Prevents accidental publishing to npm
- **"type": "module"** : Enables ES module syntax (`import/export` instead of `require/module.exports`)

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "preview": "vite preview"  
},
```

- **dev** : Starts Vite development server with hot-reload
- **build** : Creates optimized production build in `dist/` folder
- **preview** : Serves the production build locally for testing

```
"dependencies": {  
  "react": "^18.3.1",  
  "react-dom": "^18.3.1"  
},
```

- **react** : Core React library (component model, hooks, state management)
- **react-dom** : Renders React components to the browser DOM

```
"devDependencies": {  
  "@vitejs/plugin-react": "^4.3.4",  
  "vite": "^6.0.3"  
}
```

- **vite** : Build tool — extremely fast bundler using native ES modules
- **@vitejs/plugin-react** : Adds React support to Vite (JSX transform, fast refresh)

17. Frontend — Vite Configuration

File: `frontend/vite.config.js`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000,
    proxy: {
      '/api': {
        target: 'http://localhost:8000',
        changeOrigin: true,
      },
    },
  },
});
```

- **plugins: [react()]** : Enables JSX/TSX compilation and React Fast Refresh (instant updates without losing state)
 - **server.port: 3000** : Dev server listens on port 3000
 - **proxy: { '/api': ... }** : In development, requests to `/api/*` are proxied to the backend at `localhost:8000` . This avoids CORS issues during development.
-

18. Frontend — index.html

File: `frontend/index.html`

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>AI Contract Analyzer</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
```

```
</body>
</html>
```

- **<div id="root">** : The DOM element where React mounts the entire application
 - **type="module"** : Loads the script as an ES module (enables `import` statements)
 - **Vite uses this as the entry point** — it finds `/src/main.jsx` and bundles everything from there.
-

19. Frontend — main.jsx (Entry Point)

File: `frontend/src/main.jsx`

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

- **ReactDOM.createRoot(...)** : Creates a React 18 root (concurrent mode capable)
 - **document.getElementById('root')** : Finds the `<div id="root">` from index.html
 - **.render(...)** : Renders the component tree into the DOM
 - **<React.StrictMode>** : Development-only wrapper that warns about deprecated patterns and double-invokes effects to catch bugs
-

20. Frontend — App.jsx (Main Component)

File: `frontend/src/App.jsx`

API Base URL Detection

```
const API_BASE = window.location.hostname === 'localhost' && window.location.port === '' ? 'http://localhost:8000' : '';
```

- **Purpose:** If running locally on port 3000 (development), send API requests to `localhost:8000`. In production (served from same origin), use relative URLs.

State Management (React Hooks)

```
const [messages, setMessages] = useState([]); // Chat history array
const [question, setQuestion] = useState(""); // Current input text
const [uploading, setUploading] = useState(false); // Upload in progress flag
const [uploadStatus, setUploadStatus] = useState(""); // Upload status message
const [streaming, setStreaming] = useState(false); // AI response streaming flag
const chatEndRef = useRef(null); // Reference to scroll anchor
```

- **useState(initial)** : React hook — returns [currentValue, setterFunction]. Re-renders component when value changes.
- **useRef(null)** : Creates a persistent reference to a DOM element (doesn't cause re-renders).

Auto-Scroll Effect

```
useEffect(() => {
  chatEndRef.current?.scrollIntoView({ behavior: 'smooth' });
}, [messages]);
```

- **useEffect(..., [messages])** : Runs every time `messages` array changes
- **chatEndRef.current?.scrollIntoView** : Scrolls to the invisible div at the bottom of the chat — keeps latest messages visible

File Upload Handler

```
const handleUpload = async (e) => {
  const file = e.target.files[0]; // Get the selected file
  const formData = new FormData(); // Create multipart form data
  formData.append('file', file); // Attach file with key 'file'
```

```
const response = await fetch(`${API_BASE}/api/upload`, {
  method: 'POST',
  body: formData, // Browser auto-sets Content-Type with boundary
});
const result = await response.json(); // Parse JSON response
};
```

Streaming Chat Handler (Most Important)

```
const response = await fetch(`${API_BASE}/api/chat/stream`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ contract_id: 1, user_id: 101, question: userMessage.content }),
});
```

- **Purpose:** Sends the question to the streaming endpoint.

```
const reader = response.body.getReader();
const decoder = new TextDecoder();
```

- **response.body.getReader()** : Gets a `ReadableStream` reader — allows reading the response chunk by chunk as it arrives.
- **TextDecoder** : Converts raw bytes to text.

```
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  buffer += decoder.decode(value, { stream: true });
}
```

- **Purpose:** Read loop — keeps reading chunks until the stream ends (`done === true`).
- **{ stream: true }** : Tells the decoder this is a partial stream (don't finalize multi-byte characters).

```
const lines = buffer.split('\n');
buffer = lines.pop() || '';
for (const line of lines) {
  if (line.startsWith('data:')) {
```

```

const data = line.slice(5);
// Append token to the last message
setMessages((prev) => {
  const updated = [...prev];
  const last = updated[updated.length - 1];
  updated[updated.length - 1] = { ...last, content: last.content + data };
  return updated;
});
}
}

```

- **SSE parsing:** Server-Sent Events send data as `data:token\n` lines
 - **`line.slice(5)`** : Removes the `data:` prefix to get the actual token text
 - **State update:** Appends each token to the last message (the AI's response), causing a re-render that shows the text appearing character by character
-

21. Frontend — Dockerfile

File: `frontend/Dockerfile`

```

FROM node:20-alpine AS build
WORKDIR /app
COPY package.json package-lock.json* ./
RUN npm install
COPY . .
RUN npm run build

```

- **Stage 1:** Install dependencies and build the React app into static files (`dist/`)
- **`package-lock.json*`** : The `*` makes it optional (won't fail if it doesn't exist)

```

FROM node:20-alpine
WORKDIR /app
RUN npm install -g serve
COPY --from=build /app/dist ./dist
EXPOSE 3000
CMD ["serve", "-s", "dist", "-l", "3000"]

```

- **Stage 2:** Uses `serve` to host static files

- `-s` : Single-page app mode (all routes serve `index.html`)
 - `-l 3000` : Listen on port 3000
-

22. Complete Data Flow: How the Contract Analyzer Fetches Correct Results

Flow 1: Document Upload

User selects PDF Browser Frontend (React)

POST /api/upload (multipart/form-data)

Backend: UploadController receives FilePart

Backend: DocumentIngestionService.ingestDocument()

Step 1: Save to temp file

Step 2: TikaDocumentReader extracts text from PDF

Step 3: TokenTextSplitter breaks into chunks (1000 chars, 200 overlap)

Step 4: Add metadata (contract_id=1, user_id=101) to each chunk

Step 5: DELETE old vectors for this contract from PostgreSQL

Step 6: vectorStore.add(chunks)

For each chunk: Send text to Ollama nomic-embed-text

Ollama returns 768-dimension vector

INSERT into vector_store (id, content, metadata, embedding)

Return success response Frontend shows "X chunks indexed"

Flow 2: Question Answering (RAG)

User types question Frontend (React)

POST /api/chat/stream (JSON body)

Backend: ChatController RagService.streamResponse()

Step 1: Build filter expression (contract_id=1 AND user_id=101)

Step 2: SearchRequest.query("What are payment terms?").withTopK(3)

Step 3: vectorStore.similaritySearch(searchRequest)

 Send question to Ollama nomic-embed-text get query vector

 PostgreSQL: Compare query vector against all stored vectors
 using HNSW index + cosine similarity

 Filter results by metadata (contract_id, user_id)

 Return top 3 most similar chunks

Step 4: Join chunk texts into context string

Step 5: Build system prompt:

 "You are a contract analyst..."

 CONTEXT: [chunk1] --- [chunk2] --- [chunk3]"

Step 6: chatClient.prompt().system(prompt).user(question).stream().content()

 Send to Ollama llama3 model

 Receive tokens one by one (streaming)

Step 7: Each token flows back as SSE: "data:The\ndata: payment\ndata: terms\n..."

Frontend: ReadableStream reader reads each chunk

React state update: Append each token to message UI re-renders

User sees answer appearing word by word in real-time

23. Key Concepts for Interview

1. What is Reactive Programming (WebFlux)?

- **Traditional (Servlet)**: One thread per request. 1000 concurrent requests = 1000 threads.
- **Reactive (WebFlux)**: Non-blocking I/O with few threads. Uses event loop + callbacks. `Mono` = 0 or 1 result, `Flux` = 0 to N results.

2. What is a Vector Embedding?

A mathematical representation of text as a list of numbers (768 floating-point values). Semantically similar texts produce similar vectors. Example:

- “payment deadline” [0.12, -0.34, 0.56, ...]
- “when to pay” [0.11, -0.33, 0.55, ...] (very similar!)
- “cat food” [0.89, 0.12, -0.67, ...] (very different)

3. What is Cosine Similarity?

Measures the angle between two vectors. Range: -1 to 1. Higher = more similar.

- 1.0 = identical meaning
- 0.0 = unrelated
- Used instead of Euclidean distance because it's scale-independent.

4. What is HNSW?

Hierarchical Navigable Small World — a graph-based algorithm for Approximate Nearest Neighbor (ANN) search. Instead of comparing against every vector ($O(n)$), it navigates a graph structure for $O(\log n)$ search. Trade-off: slightly less accurate but dramatically faster.

5. Why Multi-Stage Docker Build?

- Build stage: Has JDK + Maven (~700MB) — compiles code
- Runtime stage: Has only JRE (~300MB) — runs the JAR
- Result: Smaller image, faster deployment, smaller attack surface.

6. Why `subscribeOn(Schedulers.boundedElastic())` ?

WebFlux uses a small number of event loop threads (typically CPU cores × 2). Blocking these threads (e.g., during PDF reading or DB queries) would freeze the entire server.

`boundedElastic()` provides a separate thread pool for blocking operations.

7. What is the ChatClient Fluent API?

Spring AI's modern API for interacting with LLMs:

```
chatClient.prompt()
    .system("instructions") // Set system context
    .user("question")      // Set user input
    .stream()              // Enable streaming
    .content();             // Get text content as Flux<String>
```

8. What is Server-Sent Events (SSE)?

A standard for streaming data from server to client over HTTP:

- Response header: `Content-Type: text/event-stream`
- Data format: `data: value\n\n`
- Connection stays open until complete
- One-directional (server → client only)
- Simpler than WebSockets for streaming use cases

9. Why JSONB for Metadata?

- **JSON**: Stored as text, parsed on every access
- **JSONB**: Stored as binary, pre-parsed, supports GIN indexes
- Enables efficient filtering: `WHERE metadata->>'contract_id' = '1'`

- Flexible schema — can store any key-value pairs without altering the table structure

10. What is Spring AI's VectorStore Abstraction?

An interface that hides the implementation details:

- `vectorStore.add(documents)` generates embeddings + stores them
- `vectorStore.similaritySearch(request)` embeds query + finds similar docs
- Implementations: `PgVectorStore`, `ChromaVectorStore`, `MilvusVectorStore`, etc.
- Switching databases requires only configuration changes, not code changes.

End of Document